C++中 vector 的全面用法及注意事项

我来给你**通俗、系统地讲解 C++ `vector` 的用法**，以及**使用 vector 动态数组时的核心注意事项**（这是新手最容易踩坑的地方）。

`vector` 是 C++ STL 标准库中的**动态数组**，完美替代普通数组和 `new[]` 手动创建的数组，**自动管理内存、自动扩容、安全易用**，是 C++ 开发最常用的容器！

一、vector 基础

1. 必备头文件

Code block

```
#include <vector>   // vector 核心头文件
#include <iostream>
using namespace std;
```

2. vector 是什么？

- 本质：**可变长的动态数组**
- 优势：不用 `new/delete`、不用关心内存泄漏、不用固定大小、自动扩容
- 支持所有数据类型：`int` / `double` / `char` / `string` 甚至自定义类型

二、vector 定义与初始化（最常用 5 种）

Code block

```
// 1. 空 vector（最常用）
vector<int> v1;

// 2. 指定大小，默认初始值为 0
vector<int> v2(5);   // 5个int元素，值都是0

// 3. 指定大小 + 统一初始值
vector<int> v3(5, 10); // 5个元素，值都是10

```

```
// 4. 初始化列表（直接赋值）
vector<int> v4 = {1,2,3,4,5};

// 5. 拷贝另一个 vector
vector<int> v5 = v4;
```

三、vector 核心常用操作（增删改查）

1. 访问元素

Code block

```
vector<int> v = {10,20,30};

// 方式1：[] （和普通数组一样，不检查越界）
cout << v[0];  // 10

// 方式2：at() （检查越界，越界会报错，更安全）
cout << v.at(1); // 20

// 访问首尾元素
cout << v.front(); // 第一个
cout << v.back();  // 最后一个
```

2. 添加元素

Code block

```
vector<int> v;
// 尾部添加（最常用）
v.push_back(10);
v.push_back(20);
// 结果：[10,20]
```

3. 删除元素

Code block

```
// 删除最后一个元素
v.pop_back();

```

```
// 清空所有元素
v.clear();

// 判断是否为空
v.empty(); // 空返回true
```

4. 获取大小 / 容量

Code block

```
v.size();     // 实际元素个数（重点）
v.capacity();// 总容量（自动分配的空间）
```

四、vector 遍历（3种简单方式）

Code block

```
vector<int> v = {1,2,3,4,5};

// 方式1：普通for循环（最通用）
for(int i=0; i<v.size(); i++){
    cout << v[i] << " ";
}

// 方式2：范围for（C++11，极简）
for(int num : v){
    cout << num << " ";
}
```

五、🔥 重点：vector 使用注意事项（新手必背）

这是你问的核心！**普通数组没有这些问题，vector 独有的坑**：

1. 绝对不要用 `sizeof` 求长度！

普通数组 `sizeof(arr)` 能算总大小，但 **vector 是对象，sizeof 只能得到对象大小**，必须用 `v.size()`！

Code block

```
vector<int> v(5);
// 错误
cout << sizeof(v);  // 不是20（5*4）！
// 正确
cout << v.size();   // 5
```

2. `[]` 不检查越界，`at()` 会检查

- `v[100]`：越界直接程序崩溃，不报错
- `v.at(100)`：越界抛出异常，方便调试

开发推荐用 `at()`

3. 不要提前访问未赋值的位置

vector 初始为空时，**不能直接用 `v[0]` 赋值**，必须先 `push_back` 或指定大小：

Code block

```
vector<int> v;
v[0] = 10; // ❌ 错误！空数组无元素，越界
v.push_back(10); // ✅ 正确
```

4. `size`（实际元素）≠ `capacity`（总容量）

vector 自动扩容，**容量 ≥ 大小**，扩容时会重新分配内存，这是正常现象，不用管。

5. 自动释放内存，无需手动删除

vector 离开作用域（比如函数结束、main结束），**自动释放内存**，不用 `delete[]`，完全避免内存泄漏！

6. 初始化大小后，`push_back` 会追加

Code block

```
vector<int> v(3); // [0,0,0]
v.push_back(10); // [0,0,0,10]  （不是覆盖！）
```

7. 存储字符串/char 也可以

Code block

```
vector<string> v = {"apple", "banana"};
```

```
vector<char> v = {'a','b','c'};
```

六、完整可运行代码（整合所有用法）

Code block

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    // 1. 初始化
    vector<int> v;

    // 2. 添加元素
    v.push_back(10);
    v.push_back(20);
    v.push_back(30);

    // 3. 打印大小
    cout << "元素个数: " << v.size() << endl;

    // 4. 遍历
    cout << "元素: ";
    for (int i = 0; i < v.size(); ++i) {
        cout << v.at(i) << " ";
    }
    cout << endl;

    // 5. 修改元素
    v[0] = 100;

    // 6. 删除最后一个
    v.pop_back();

    // 7. 清空
    v.clear();
    cout << "清空后是否为空: " << v.empty() << endl;

    return 0;
}
```

运行结果

Code block

```
元素个数：3
元素：10 20 30
清空后是否为空：1
```

七、vector vs 普通数组 vs new 动态数组

方式	大小	内存管理	安全性	推荐度
普通数组	固定	自动	低	不推荐
new 动态数组	可变	手动delete	极低	不推荐
vector	可变	自动	高	✅ 首选

总结

1. **vector = 安全、自动扩容、自动释放的动态数组**
2. 核心操作：`push_back` 添加、`size` 获取长度、`[]/at` 访问
3. **核心注意事项：**
   - 用 `v.size()` 求长度，不用 `sizeof`
   - 空数组不能直接用 `[]` 赋值
   - 不用手动释放内存
   - `at()` 比 `[]` 更安全

（注：文档部分内容可能由 AI 生成）